

MOBILE PAYMENT SYSTEMS AND THE VULNERABILITIES

Kranthi K Chinnakotla

Introduction:

This study is about vulnerabilities in using mobile phones to make payment using online payment systems and payment SDKs available in the market. The main focus is on two mobile operating systems, iOS and Android. There are more than 100 popular payment SDKs available currently.

Only couple of these SDKs like Stripe and PayPal are considered for this study, as most SDKs follow the similar process. This study covers the various payment services provided by PayPal, Stripe and implementing those services in the mobile devices.

The Payment SDKs provide the security to their user by implementing Open Authorization technology, which is also called as OAuth. By implementing OAuth, the “payment services providers(PSP)” will prevent the users from sharing their user credentials with a third party application (Merchant Application), the OAuth also lets the users choose the level of access to be provided for the third party application. The users can also revoke the access at any point of time.

Most of the service providers have their client and server SDKs available, any third party application can include these SDKs in their application and these SDKs will handle the authorization and authentication process. If the developer chooses not to use the provider’s SDK then they can implement own libraries, but follow the structure and OAuth protocols defined by the service

providers. The SDKs provided by the payment service providers adopt three different processes to authorize and authenticate the users,

- i. Web Component Embedded in the mobile device.
- ii. Native browser available in the mobile device.
- iii. The provider’s application installed in the mobile device.

This study focuses on

1. OAuth implementation in mobile devices for Payment SDKs.
2. Vulnerability in mobile devices of exposing the user credentials and payment information when Payment SDKs are used.

OAuth implementation in mobile devices for Payment SDKs:

The native mobile applications(client) need to access user details from the provider’s server. Saving the client credentials in the mobile application is not safe and secure, this doesn’t enable mobile application to make a direct call to authorization server using the client credentials. So, the authorization server requires the client to register a redirect URI with the provider, so that the provider can redirect the call to user through the mobile application and get the user approval or confirmation. The user-agent to make the connections between client and the authorization server is either a native browser in the mobile, or embedded web component in the mobile (like WebView in android and UIWebView in iOS), or the

provider's mobile application installed in the same device.

In the next few sections we will discuss on the OAuth implementation using the available user-agents in mobile devices:

i. OAuth implementation using WebView or Provider's SDK:

We will be considering PayPal SDK for this section. The PayPal has mobile SDKs for Android and iOS, it also has the server SDKs in PHP, Python, Node.js and Java. In this section we will focus on Android client SDK and PHP server SDK.

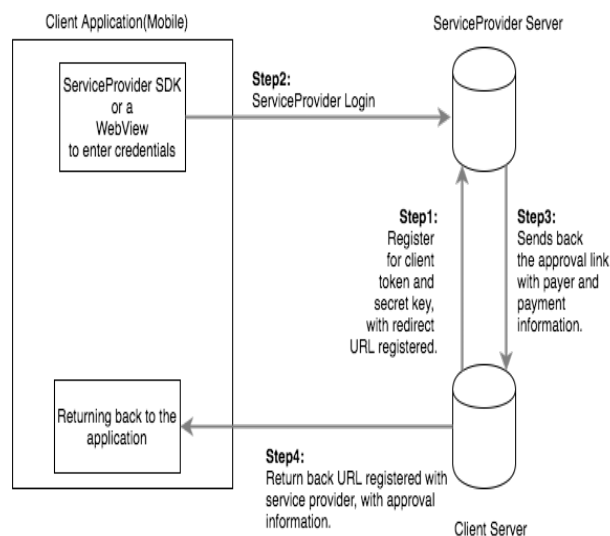


figure 1.1

The steps shown in the figure 1.1 are discussed in detail here:

Step1: The client creating the mobile application should create a merchant or business account with the service provider. In this case we consider the service provider

to be PayPal, so the client would register the application in PayPal Rest API apps section. For each client the Client ID and the Secret Key are generated which is required to authorize the application with PayPal.

After the registration the client would need to enable the required features from the PayPal website:

- Login users with PayPal, can also enable seamless checkout and let user stay login for one hour.
- Enable Payments, to accept payments from the customers using PayPal or credit card transactions.
- Create invoices for the customers.
- Send batch payments to multiple accounts, the amounts can be varied based on the recipient.
- Process swiped credit card transactions using PayPal here.

The client could also enable the redirect URL option and provide the URL which would take the control back to the mobile application and take the approval from the user/customer.

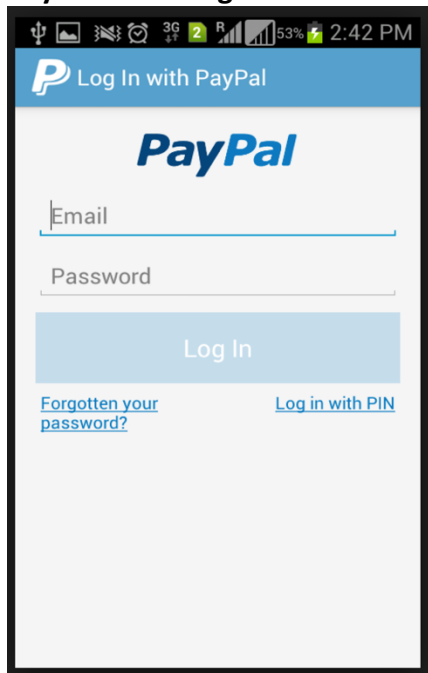
Step 2: The PayPal client SDK (Android SDK) uses the native view to get user credentials, if the client application in the android has decided not to use PayPal client SDK then then can even use WebView. The user-agent here is either WebView or PayPal SDK, depends on the choice of the client app.

The user selects the product from the client application and chooses to pay using PayPal, and as soon as the user clicks on the PayPal login button, he would be presented with WebView or Client SDK to enter the credentials. The user would enter the credentials to authenticate in to the PayPal.

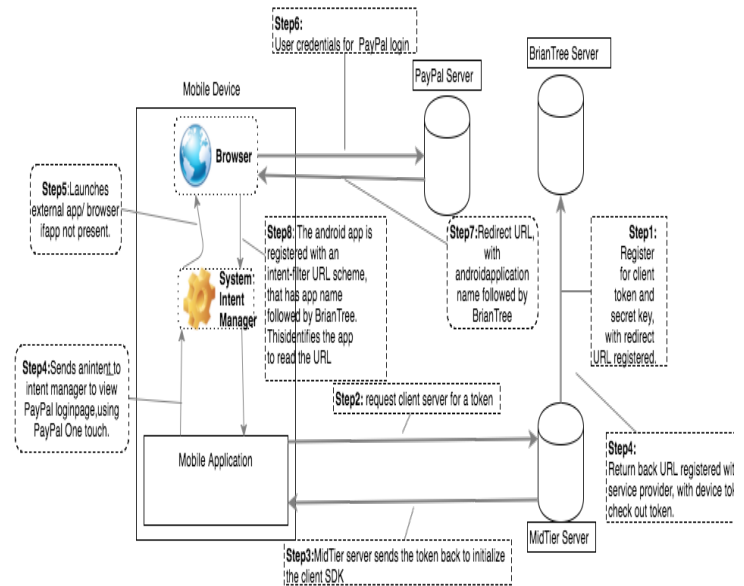
Step 3: The client server uses the Client – ID and secret key allotted to the client app, and creates PayPal objects: Transaction, Payer, Payment. Using these PayPal objects the client server could get the redirect URL (approval link) the client had registered in the PayPal website. The redirect URL is sent back to server with Payer-ID and Payment-ID appended to the URL. These Payer – IDs and Payment – IDs are used to identify the user and the transaction amount to be deducted from the user’s balance. The sale-intent type in the request from the client server to the PayPal server could either “PayPal” or “Credit Card”.

Step 4: The redirect URL is shown in the WebView of the client mobile application, this URL would take the approval from the user to apply the charges. The user will be taken back to the mobile application.

PayPal SDK using webview:



ii. OAuth implementation using native browser:



For understanding the implementation of OAuth, when the service provider website is opened external to the mobile application, we had considered Braintree SDK and PayPal SDK. Braintree is a child company of PayPal.

Step 1: The client has to create an account with BrianTree to generate access token, this token is saved in the MidTier server (Client Server).

Step 2: The client application (mobile application) sends a request to Client server to get the access token, this access token is used to initialize the BrianTree client SDK in the mobile. With the BrianTree client SDK , we will consider Android SDK for this study, the user accessing the mobile application will be able to make payments using Android Pay, Apple Pay, PayPal, Venmo and/or personal credit cards.

The Braintree client SDK provides a drop-in UI, which makes it easy for the client to add a full featured checkout with PayPal and Credit Card.

Step 3: The client server sends the token back to the mobile application, the token is used to initialize the client SDK on the mobile.

Step 4: The Braintree client SDK provides various options to login, PayPal, Venmo, Credit Card, Android Pay(Android) and Apple Pay(iOS). For this scenario we will consider PayPal option, when we click on the PayPal option from the mobile application, the mobile system tries to open the PayPal application if it is installed in the mobile device, otherwise the PayPal login is opened in the mobile native browser, in either case the Android intent manager is used. The mobile application sends an intent to the System intent manager, requesting intent manager to find any application installed in the device that could respond to the request.

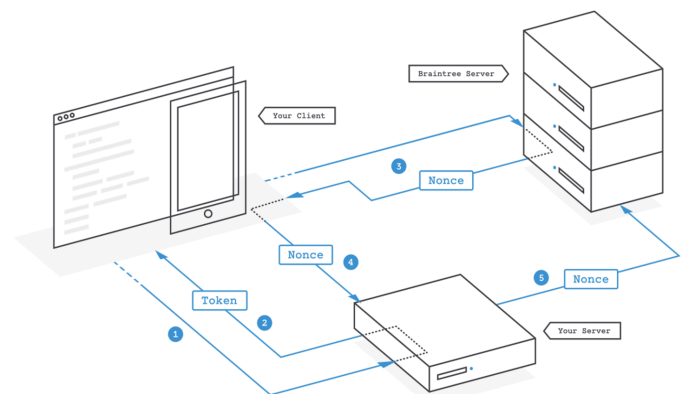
Step 5: The system intent manager tries to find an external application for PayPal that has an intent-filter to listen and respond back to the specific intent that was broadcasted by the system intent-manager. If the PayPal application is not installed in the mobile device, then the native browser would respond to the intent request broadcasted by the intent manager.

Step 6: The user would enter the login credentials for the PayPal and the request is sent to the PayPal for authentication.

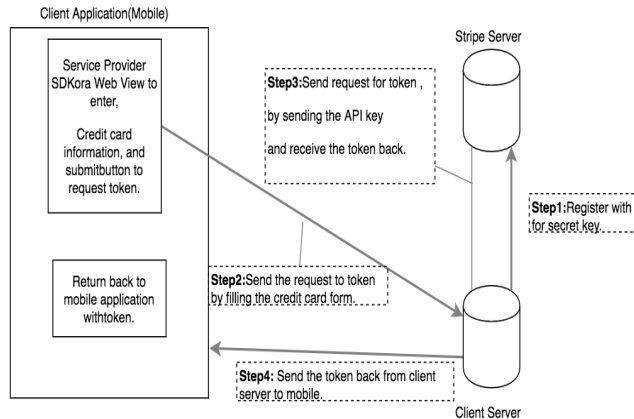
Step 7: The client should have a registered application in PayPal, with a client-id and secret key and redirect – URL. This redirect-URL is used to navigate the user back to the client mobile application from PayPal. As a response back to the user request for authentication, the PayPal returns redirect-URL by adding the Payer ID and Payment ID.

The attached IDs to the redirect-URL would let the client application deduct the right amount from the user. The redirect URL is registered in the PayPal with a specific MIME (Multi-Purpose Internet Mail Extensions) – that could enable the mobile application read the contents of the URL. The MIME for this scenario is the android application name followed by the word “Braintree”

Step 8: The Android mobile application has an intent filter with URI scheme as application name followed by “Braintree”. This had enabled the mobile application respond to the redirect URL broadcast. The intent filter to respond is added in the manifest file of the mobile application. The parameters added to the redirect URL could be read by the mobile application using the *getQueryParameters ()* method. The mobile application would request a Payment nonce with the payment information from the Brian Tree Server. The same payment nonce is sent to the client server and then the payment is authorized after the client server confirming the nonce with the Braintree server.



iii. OAuth implementation using the WebView and payment through credit card:



In this scenario we would be covering the OAuth implementation and payment procedure of the Payment system STRIPE, using Android mobile system and webview and when a credit card is used for a payment. Stripe provides a java .jar files, the client mobile application should add these jar files in their SDK to implement the Stripe SDK. If we use the STRIPE SDK, the PCI compliance when using the credit card in our application is taken care by the STRIPE. We can still try to access without the STRIPE client mobile SDK, but then the client has to take care of the PCI compliance on his own.

Step 1: The client would register the application with the Strip and obtain a secret key, which is used in the client server for authorization.

Step 2: The client mobile application, would Provide a form to fill the credit card details using a webview or a STRIPE SDK. The credit card details are then sent to the client server (PCI Compliant) and wait for the response back from the client server.

Step 3: At this point the client server would send a request to STRIPE server using the secret key obtained by client while registering for the STRIPE services. The Stripe server send the access token back to the client server.

Step 4: The client server then sends the token back to the mobile application, as a return/ response to the request sent by the mobile application.

Step 5: This token has validity and the client application uses this token to obtain services form the STRIPE, like charging the customer for payment.

To have the stripe working in the android, the minimum API level for android has to be 17. The stripe package name, com.stripe.android needs to be added in the Grade file. The Stripe also supports integrating Android Pay and Apple pay in the client application, the user could use the credit card save in the Android Pay or Apple pay to make the payments.

The token generated using the API credentials from the Brian tree server, can be used by the mid-tier servers and mobile applications to get the payment from the customers or any other features/services provided by the payment gateway (In this example by Stripe).

Images of the “STRIPE” client SDK using Android before receiving token from the server:

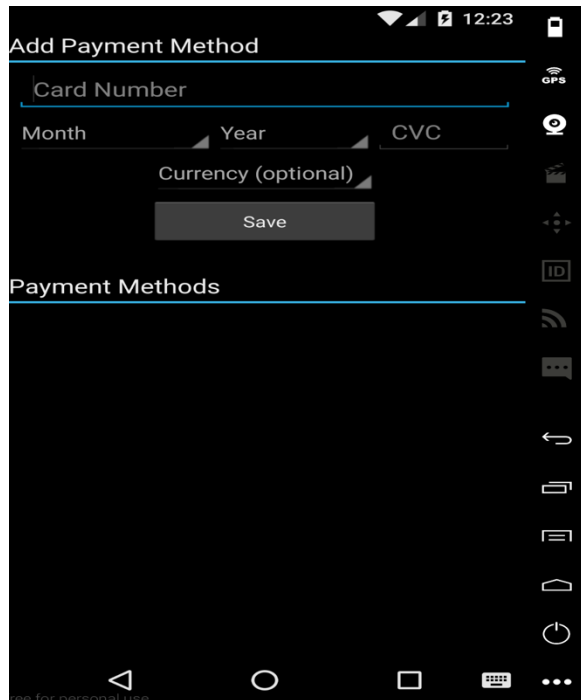
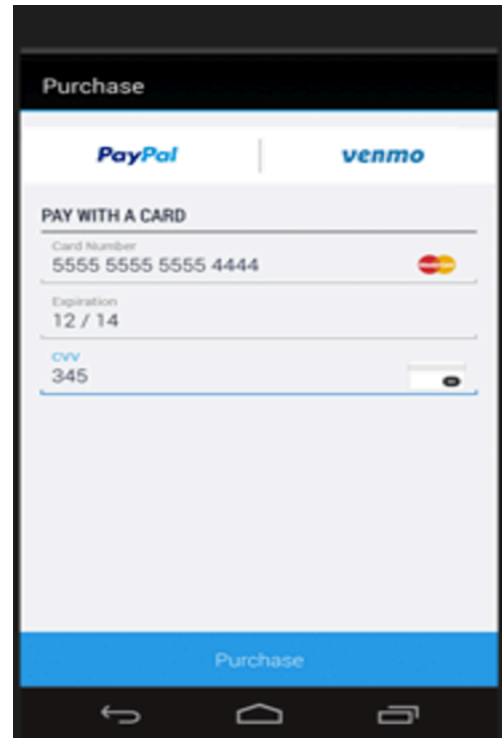


Image of BrianTree client SDK, when drop-in UI is used:



Images of the “STRIPE” client SDK using Android after receiving token from the server:

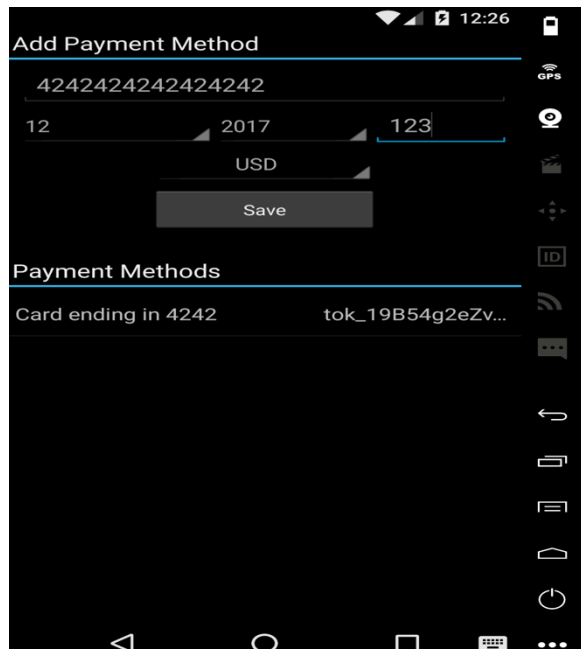
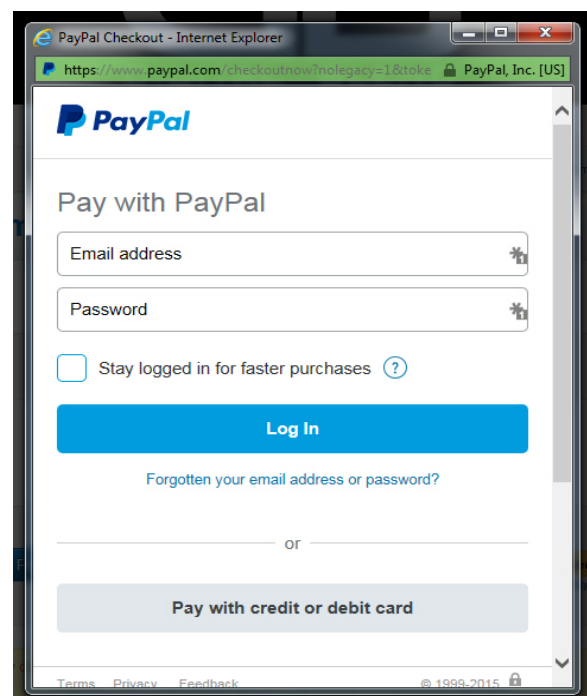


Image of PayPal login in Native browser:



We have seen using Web View, external application, external browser in using the Payment gateway. All these approaches expose the user to sever security threats. In the next few sections the paper discusses about the security incidents that could occur in each one of these scenarios. At the end we would also discuss about the malicious applications that are currently available and the impact they have on the customers.

Threats using embedded web-browser or web-view:

There is no required isolation between the user agent and the hosting mobile application. The mobile application can get the full access to the hosting webView. The hosting application could steal user credentials and also get application authorization.

To get the user credentials from the webView, the malicious app injects the JavaScript in the webView to retrieve the users credentials, the malicious app also registers JavaScript interface to send the application credentials to the hosting application.

The malicious app could also get the application access through the webView. After the user log in through the webView, the Payment Gateway application shows the required permissions, since this is running on webView interface the malicious app could change the required permissions and the user could be tricked in providing unwanted permissions to the malicious application.

Threats using System Native browser:

The external browser in the mobile device provides the required isolation between the user-agent and the hosting application. In this case the external browser which is

acting as a user-agent is an external application. To take the user from the native browser to the mobile application and use the authorization code, the callback URL is set to a specific "*Multipurpose Internet Mail Extensions*" called MIME. The client application would have registered an intent filter to read this specific MIME.

Compared to the embedded web-browser/webView, the system browser provides stronger isolation, but there is still a threat. If the malicious app has registered the same intent filter to read the MIME of the callback URL, between the intent manager and the mobile application, then access token or authorization code could be shared with the malicious app. The malicious app gets access to the Physical device and the legitimate and malicious apps both are shown for the user when callback URL is used in the native browser. The user could be easily tricked to open the malicious app.

Threat using external application:

Using an external application is similar to using a native browser, the Payment gateway resource provider application is installed in the mobile device. The resource provider application is opened and communicated back through intent-manager. The malicious app could have the same intent-filter as the payment gateway app.

Using external app to communicate to the payment gateway isolates the user-agent with the host application, compared to webView, using external app and external browser is less risky.

Sample Malicious Apps and the impact of these apps:

Flash Player App:

This application is a fake Adobe Flash Player application; the authentic adobe flash player can be installed from the Google Play store. This app asks for the below permissions:

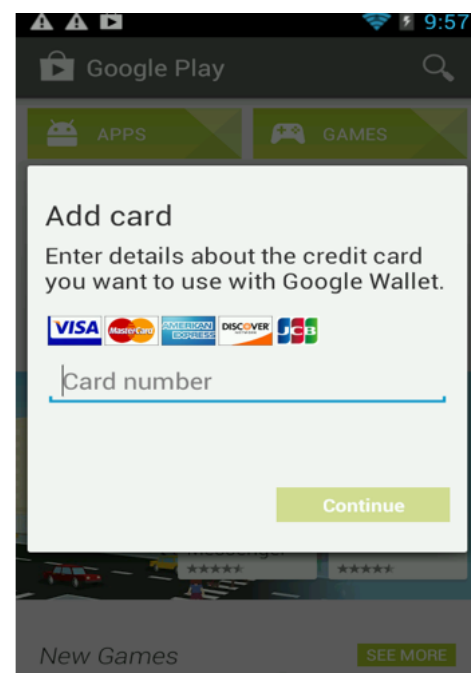
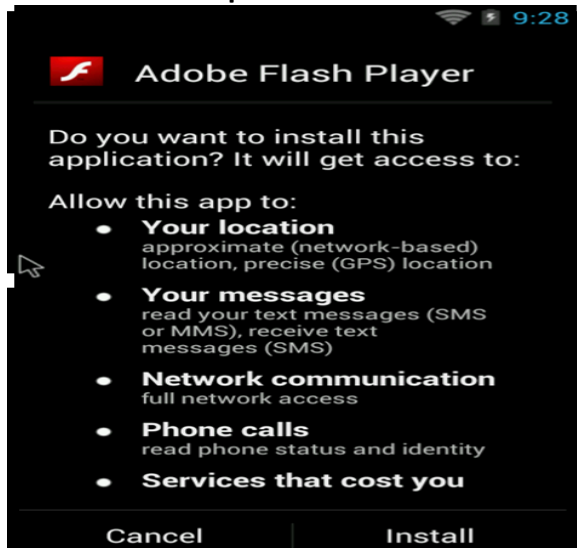
1. Location,
2. Messages,
3. Network Communications,
4. Phone calls,
5. System tools.

The flash player app also requests access for video codec, after installing itself as a service which runs in the background. Once the flash player app is installed, it targets few popular apps like Google Play services, Facebook etc. They could target the Payment gateway apps too. If any of these targeted apps are opened then the malware/fake flash player app requests for a credit card information by showing a dialog requesting credit card details.

The form to enter credit card details looks more authentic and it even checks for incorrect card numbers and the CVVs.

This credit card information is directly shared with cybercriminal servers.

Below are few pictures of how this app looks in android phones:

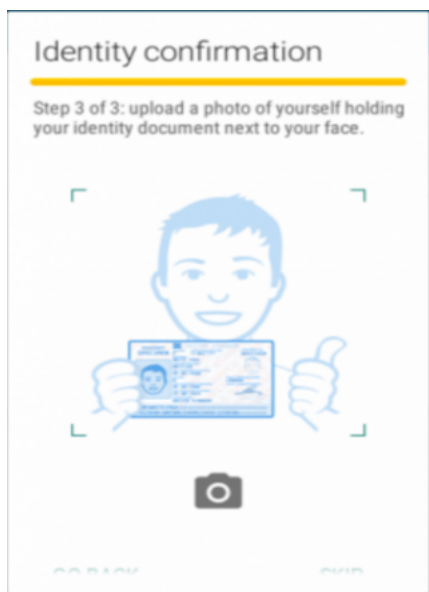


Malicious Selfie App:

There was a malicious application exposed during the recent times, it is popularly known as “Malicious Selfie App”. This malicious app was discovered by McAfee.

The malicious app once installed asks the user to enter personal information, credit card information with CVV. The app also asks the user to take a selfie with an identity document front and back part of the document. This not only allows the cyber criminals to take the credit card information, they could also take the personal identity, which could help the criminals to access the bank accounts and other social network sites of the victims.

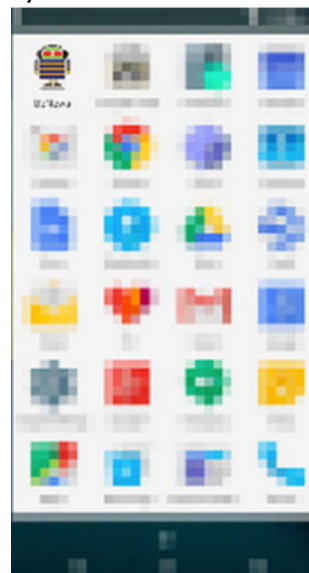
This app is not available in the google play store though, this app also asks the access to Video HD codec and takes control of the system.



Be News App:

The previous two apps were downloaded from the outside the google play store. The app we would talk about now, known as “Be News” is a malicious app available at the Google Play Store and was installed 40 times before it was identified as a malicious app and removed from the Play store.

The “Be News app” asks very minimal permissions, and this app escapes all the Google Play Store permission checks. This app doesn’t show any exploit intent and so the Google Play Store couldn’t tag this app as malicious. Once the app is installed, this application loads the code dynamically from their remote servers and takes control of the system.



When these malicious apps are installed from the play store and run. They run as a service in the background, or take control of the system completely. They could target any application and restart the device and replace the targeted application with a fake application which exactly looks like the original one. The target application could be a Payment gateway application, which takes

the user credentials and sends them to the cybercriminal servers.
This security incident was discovered by IBM research teams.

Prevention steps from installing and using malicious apps:

Following the below few simple steps could protect the mobile users from falling as a prey to the cybercriminals.

1. Avoid installing the applications that are not available in the Google Play store or Apple store.
2. Avoid rooting the mobile devices, which could expose the device to more threats.
3. While installing the applications, try to check the permissions requested by the application. If it is requesting too many permissions that are not relevant to the application's functionalities or requirements, then avoid installing those applications.
4. Try not to install an application from the play store, which are not created by a popular and known user or organization.

Conclusion:

Different Payment gateway SDKs have different OAuth implementations and security assumptions. In this study we had considered most popular SDKs and the OAuth implementations. The threats that the various OAuth implementation designs expose the mobile devices are also discussed: embedded web browser, external native browser and resource provider installed application. We have also discussed the various malicious apps that

could harm mobile devices. We have also discussed about preventive measures to follow and not be victims of the cyber crime.

References:

<https://developer.paypal.com/docs/integration/mobile/mobile-sdk-overview>
<https://github.com/paypal/PayPal-Android-SDK>
https://github.com/braintree/braintree_android
<https://developers.braintreepayments.com/start/hello-client/android/v2>
<https://stripe.com/docs/mobile/android>
<https://github.com/stripe/stripe-android>
http://liisp.uncc.edu/~mshehab/wp-content/uploads/2014/06/IEEE_MS_2014.pdf

Reference Links for the Sample Malicious Apps:

<http://blog.trendmicro.com/trendlabs-security-intelligence/fake-news-app-in-hacking-team-dump-designed-to-bypass-google-play/>

<https://threatpost.com/phony-android-flash-player-installs-banking-malware/121696/>

<https://securingtomorrow.mcafee.com/consumer/identity-protection/everyone-loves-selfies-including-malware/>

<https://securingtomorrow.mcafee.com/consumer/identity-protection/everyone-loves-selfies-including-malware/>

<https://securityintelligence.com/one-class-to-rule-them-all-new-android-serialization-vulnerability-gives-underprivileged-apps-super-status/>